

Defending the Database — K Orchids · Project Green

PostgreSQL 18 · **38 tables** · **128 indexes** · 9 migrations. The database — not the application — is the source of truth: the rules that protect money and stock are enforced **inside** the schema, so they hold even under load, double-clicks, race conditions, a second server, or an application bug.

1 · Indexing strategy & the I/O argument

Every index exists to turn a **sequential scan** (read every row, $O(N)$) into an **index lookup** (B-tree height $\approx \log_b N$, a handful of pages). They sit on the columns the app actually filters, joins and sorts on.

INDEX (REPRESENTATIVE)	TYPE	WHY IT EXISTS / WHAT BREAKS WITHOUT IT
<code>products(name) · sku</code>	GIN trigram	Fuzzy catalogue search (<code>ILIKE '%phal%'</code>). Without it every keystroke is a full table scan.
<code>orders(buyer_id, status)</code>	B-tree composite	Buyer dashboards & admin queues. Without it Postgres scans all orders and discards 930 of 958 rows (measured below).
<code>invoices(due_date) WHERE status IN(PENDING,PARTIALLY_PAID,OVERDUE)</code>	Partial B-tree	Aging / overdue cron. Partial = tiny index of only unpaid rows; without it the nightly job scans all invoices.
<code>payments(invoice_id)</code>	B-tree (FK)	Balance is recomputed per invoice. Without it, every recompute scans all payments.
<code>auth_sessions(refresh_token_hash)</code>	B-tree	Hot path — validated on every token refresh. Without it, each request scans the whole sessions table.
<code>stock_movements(product_id) · (ref_table, ref_id)</code>	B-tree	Ledger look-ups by product / source document. Append-only ledger grows fast → index is essential.
<code>login_history(email, occurred_at) WHERE success=false</code>	Partial B-tree	Brute-force detection reads only recent <i>failed</i> attempts — partial index keeps it small & fast.
<code>UNIQUE(invoice_id, method, reference)</code>	Unique	Dual purpose: fast lookup <i>and</i> idempotency — a repeated payment can't be inserted twice.

Without the index — the I/O cost (measured, then scaled).

```
-- orders WHERE buyer_id=? AND status='APPROVED' (EXPLAIN ANALYZE, BUFFERS)
WITH index → Bitmap Index Scan idx_orders_buyer_status · heap blocks: 5 · 28 rows
WITHOUT idx → Seq Scan · reads all 18 pages · Rows Removed by Filter: 930 of 958
```

At today's demo scale (958 orders $\approx$ 18 data pages) both are sub-millisecond, but the seq scan already throws away **97%** of what it reads. The cost is **linear**: a sequential scan reads `relpages` every time; a B-tree reads $\approx$ `height + matched-pages`.

Scaling math. At **1,000,000 orders** (~150 MB $\approx$ 18,500 pages of 8 KB), an unindexed buyer-dashboard query reads **all ~18,500 pages (~150 MB)** per request. The composite B-tree (height $\approx$ 3) answers in **~3–5 page reads (~40 KB)** — roughly a **3–4 orders-of-magnitude** reduction in I/O, and it stays flat as the table grows.

2 · Functions, triggers & sequences — names & why

OBJECT	WHERE	WHY IT EXISTS / WHAT FAILS WITHOUT IT
<code>set_updated_at()</code>	BEFORE UPDATE trigger on every mutable table	Stamps <code>updated_at=NOW()</code> automatically. Without it, "last modified" drifts and relies on app code remembering — it won't.
<code>prevent_mutation()</code>	BEFORE UPDATE/DELETE on <code>audit_logs</code> , <code>stock_movements</code> , <code>price_history</code>	RAISEs an exception → makes the ledgers append-only at the engine level , independent of the connecting DB role. Without it the audit trail is editable and worthless as evidence.
<code>order_number_seq</code> , <code>invoice_number_seq</code> , <code>payment_number_seq</code>	SEQUENCES	Gap-tolerant, race-free document numbers. Without a sequence, two concurrent inserts can compute the same "next number" → duplicate invoice/order numbers.
<code>pg_trgm</code> / <code>citext</code> / <code>pgcrypto</code>	Extensions	Trigram search, case-insensitive unique emails, and <code>gen_random_uuid()</code> primary keys.

3 · Edge cases — scenario → mechanism → guarantee

SCENARIO	WHAT WE USED	GUARANTEE
Two people change the same thing at once — two admins approve the last 50 units simultaneously.	Transaction + row lock (SELECT ... FOR UPDATE): the second request waits, then re-reads the real availability.	One approval succeeds; the other is refused with 409 . No overselling.
Stock driven below what's reserved by a burst of orders.	CHECK constraint reserved_qty <= stock_qty — reservation bumps reserved_qty , not physical stock.	Impossible stock states cannot be stored, even with an app bug.
A payment sent twice — buyer double-clicks Pay, or the gateway notifies twice.	UNIQUE (invoice_id, method, reference) idempotency guard.	The repeat is the same payment → no double charge.
An order set to an impossible status.	CHECK enum + a guarded state machine (PENDING_APPROVAL→APPROVED→...).	Invalid / skipped transitions are rejected before storage.
The money must always add up — adjustments, partial & over-payments on one invoice.	Computed balance total + adjustments - paid under lock; overpayment → 422 ; large reversals need a second approver.	Balances always correct; big refunds can't be done alone.
Same order approved twice concurrently.	UNIQUE invoices.order_id (one invoice per order).	The second transaction rolls back — no duplicate invoice or reservation.
Someone edits history or reaches another buyer's data.	Append-only triggers + ownership scoping by trade-account id.	Audit trail is tamper-proof; each role sees only its own records.

4 · Security model

Authentication

- Passwords hashed with bcrypt (never stored in plaintext).
- JWT short-lived access token + rotating refresh token (reuse-detection revokes the session family).
- Refresh token in an httpOnly cookie; tokens stored hashed in auth_sessions .
- Brute-force logout — 5 fails / 15 min; every attempt logged to login_history .
- Email verification & one-time password-reset tokens (hashed, expiring).

Authorisation (RBAC)

- 5 roles × ~44 fine-grained permissions via role_permissions ; enforced by requirePermission() .
- Separation of duty: placer ≠ approver; price-changer ≠ price-approver; large reversals need two people.
- IDOR closed — every buyer query is scoped to the caller's trade-account id, not the user id.
- Optional time-boxed access windows per role.

Data & transport

- No SQL injection — all queries are parameterised (\$1, \$2); input validated with Zod.
- Append-only audit of every sensitive action; secrets redacted (recursive, regex) before logging.
- Idempotency on payments; two-person control on high-value reversals.
- helmet security headers, strict CORS, per-route rate limiting, request-id tracing.
- Secrets in env only — .env is git-ignored; a leaked key in history was rotated, not just deleted.
- File uploads via Cloudinary signed URLs.

Audited & verified

- 30 / 30 security & integrity findings remediated.
- 11 / 11 logic-verification assertions green.

38
TABLES

128
INDEXES

3
APPEND-ONLY LEDGERS

30/30
FINDINGS FIXED

11/11
CHECKS GREEN

In one line: the schema is the contract. Constraints, locks, unique keys, sequences, triggers and RBAC make correct money and stock the default — guaranteed by the engine, not merely hoped for by the application.